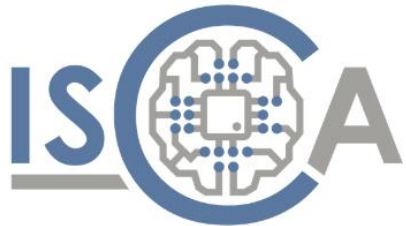


OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page I / 29



Author	Harry Koutsourelakis
Reviewer	George Kornaros
Last edit	11/03/2025

OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page II / 29

Table of Contents

Openthread	4
Why Openthread.....	4
Where Is OpenThread Used?	5
Network Components & Roles	6
OpenThread Border Router (OTBR)	6
Thread Network Devices.....	6
Tools	8
Nordic’s NRF Zephyr SDK	8
nRF Connect for Desktop	8
STM32CubeIDE.....	8
How-to and Firmware	9
nRF52840 Firmware Guide	10
nRF7002DK & nRF5340DK Firmware	22
What is OpenThread CLI Firmware?	22
Key Features of OpenThread CLI Firmware	22
Build and Flash	24
OpenThread Border Router Setup	25

OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page III / 29

OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 4 / 29

Openthread

OpenThread is an open-source implementation of the Thread networking protocol, designed for low-power, wireless mesh networks. This document outlines the high-level topology of a Thread network using an OpenThread Border Router (OTBR) setup on a Linux laptop with various development boards.

Thread itself is based on IEEE 802.15.4, the same low-power wireless standard used by Zigbee. However, instead of relying on proprietary communication methods, Thread uses IPv6, making it easier to integrate with the internet and existing networks. OpenThread brings these benefits in a lightweight, flexible package that can run on a variety of microcontrollers and wireless chips.

Why Openthread

At its core, Thread is based on IEEE 802.15.4, the same wireless standard used by Zigbee and other low-power networks. However, what sets Thread apart is its use of IPv6, thanks to 6LoWPAN (IPv6 over Low-Power Wireless Personal Area Networks).

This approach offers several key benefits:

Seamless Internet Integration: Unlike Zigbee, which requires translation to work with IP-based networks, OpenThread devices can communicate natively using IPv6, making integration with cloud services and internet-connected devices much simpler.

No Proprietary Gateways: Many IoT protocols, like Z-Wave and Zigbee, require a dedicated hub to translate their protocols into IP-based communication. OpenThread eliminates this dependency by allowing direct IP-based connectivity via a Thread Border Router.

Flexible Network Topology: OpenThread enables self-healing mesh networks, where devices can relay data between each other, extending coverage without needing a central hub. If one device fails, the network dynamically reroutes traffic through other nodes, improving reliability.

OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 5 / 29

Low Power & Efficiency: Designed with energy efficiency in mind, OpenThread allows battery-powered devices like sensors, smart locks, and remote monitoring systems to operate for years on minimal power.

Open-Source & Multi-Vendor Support: OpenThread is not tied to a single manufacturer, which encourages broader industry adoption and interoperability between devices from different brands. Major semiconductor companies like Nordic Semiconductor, Silicon Labs, NXP, and Texas Instruments support OpenThread, offering hardware solutions optimized for Thread networks.

Where Is OpenThread Used?

With its open-source nature, native IPv6 support, and strong industry backing, OpenThread is positioned as one of the most future-proof and flexible solutions for IoT networking. Its integration into Matter, the universal smart home standard, further solidifies its role as a key technology for next generation connected devices.

OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 6 / 29

Network Components & Roles

The network consists of the following components:

OpenThread Border Router (OTBR)

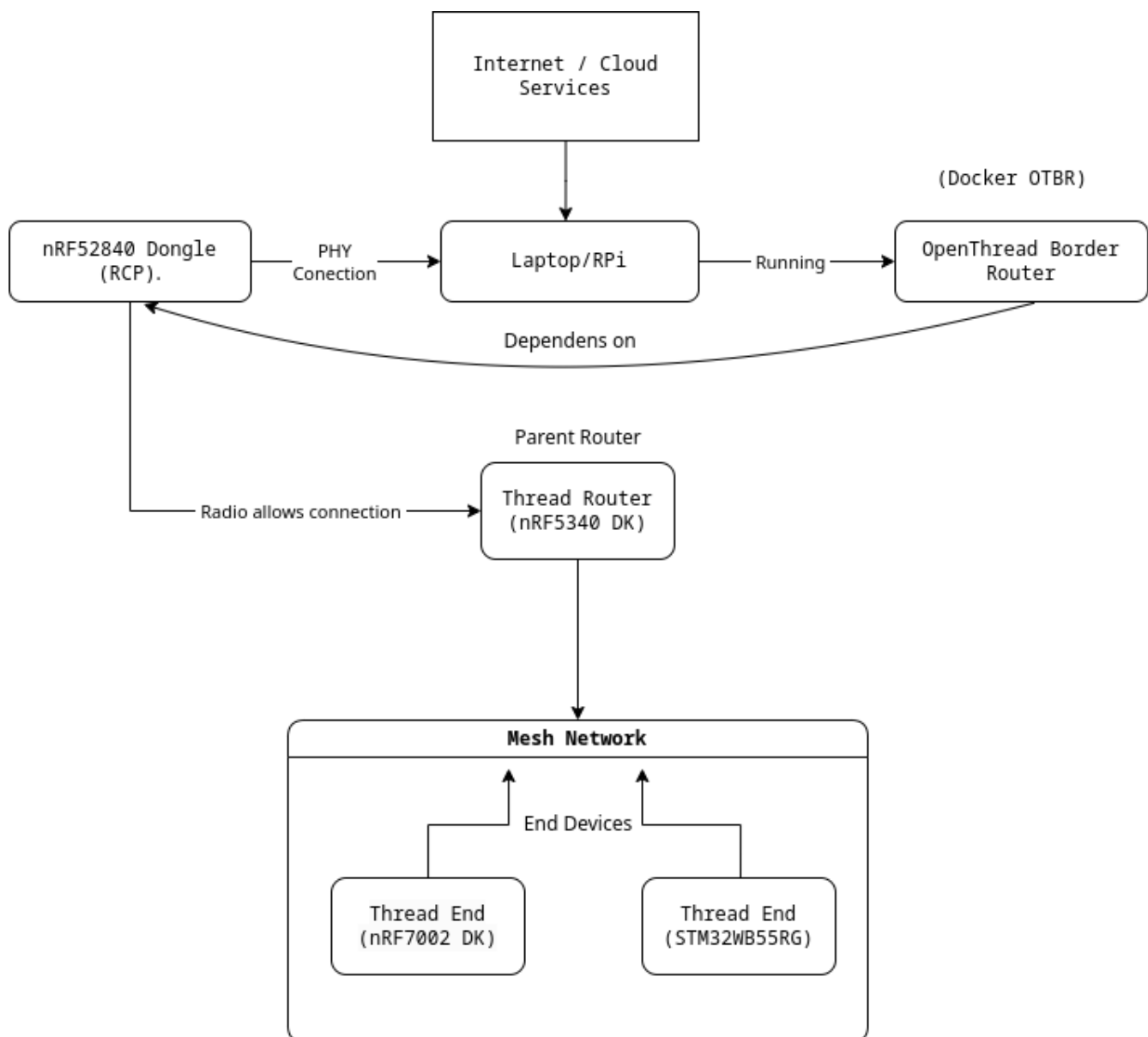
- Device: Linux Laptop running OTBR in Docker
 - Function: Acts as a gateway between the Thread network and external IP-based networks (Wi-Fi, Ethernet, Internet)
 - Thread Connection: Uses an nRF52840 Dongle as a Radio Co-Processor (RCP) for Thread communication

Thread Network Devices

- nRF52840 Dongle (RCP)
 - Role: Radio interface for OTBR
 - Function: Provides Thread connectivity for the OTBR but does not participate in routing
- nRF5340 DK (Thread Leader/Router)
 - Role: Thread Leader and Router
 - Function: Manages the network structure and forwards packets between Thread devices
- nRF7002 DK (End device in the Mesh Network)
 - Role: Router or optional secondary Border Router
 - Function: Extends network coverage and forwards messages within the Thread mesh

OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 7 / 29

- STM32WB55RG (End device in the Mesh Network)
 - Router or End Device
 - Function: Connects to the Thread mesh, may relay messages if acting as a Router.



OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 8 / 29

Tools

Nordic's NRF Zephyr SDK

Nordic's Zephyr SDK is a library of drivers and configuration files that are developed on top of the vanilla Zephyr SDK. On top of that they have developed an extension for the Visual Studio Code, which offers visual representation of the hardware layer of each device, like any other IDE (e.g. STM32CubeIDE, Microchip's MPLabX, etc.)

The development for the Nordic devices (nRF52840 Radio dongle, nRF7002DK and nRF5340DK) can be done in both Windows and Linux, since the Zephyr OS offers cross-compatibility.

In VSCode, it is simple enough to install the **nRF Connect for VS Code** extension, install the toolchain and start developing. A full guide is offered in Nordic's website:

<https://docs.nordicsemi.com/bundle/nrf-connect-vscode/page/index.html>

nRF Connect for Desktop

This cross-platform application offers many tools that work well with the Nordic devices. The one we need is the **Programmer**, in order to flash the nRF52840 dongle with the correct firmware, since it doesn't come with a dedicated hardware debugger, so it can't be flashed via the VSCode extension. But, the firmware will be build with the VSCode extension.

<https://www.nordicsemi.com/Products/Development-tools/nRF-Connect-for-Desktop>

STM32CubeIDE

The STM32CubeIDE will be used to develop the applications for the STM32WB boards.

OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 9 / 29

How-to and Firmware

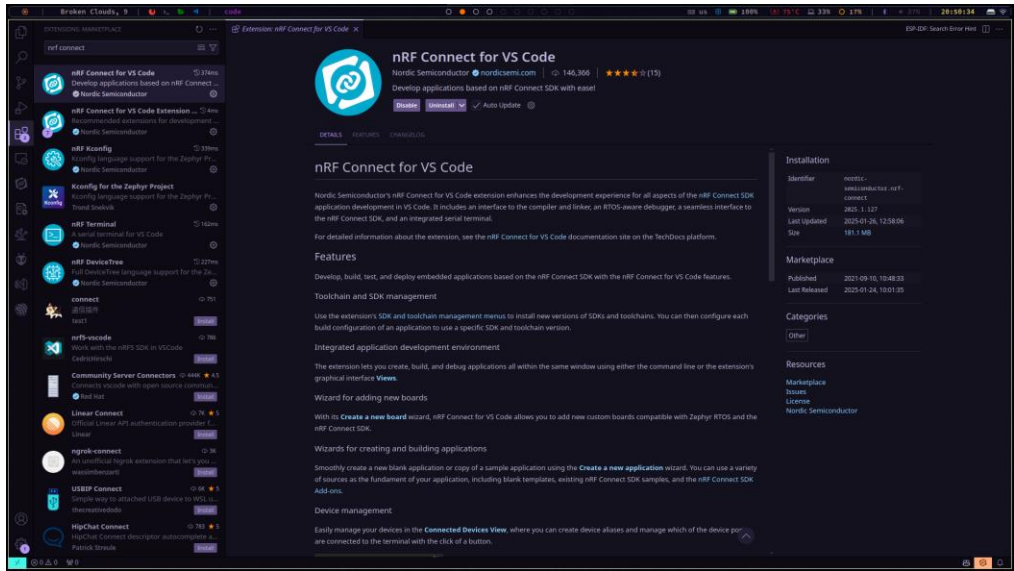
First, we need to build and flash the firmware for the nRF52840 radio dongle. The nRF52840 SoC housed in the radio dongle will be used as a radio interface for the IEEE 802.15.4 (Open/Thread), which will be connected via USB to a host machine (Raspberry Pi or other Linux machine).

The firmware used is named **Coprocessor** and is offered by many manufacturers. The name comes from the fact that it is a **co-processor firmware**, designed to work alongside a host system that handles higher-level Thread networking tasks. The nRF52840 itself primarily manages **low-level radio communication** while offloading complex processing to a separate host (like a Linux system or a microcontroller). The firmware comes into 2 types, we use the second, the **RCP**:

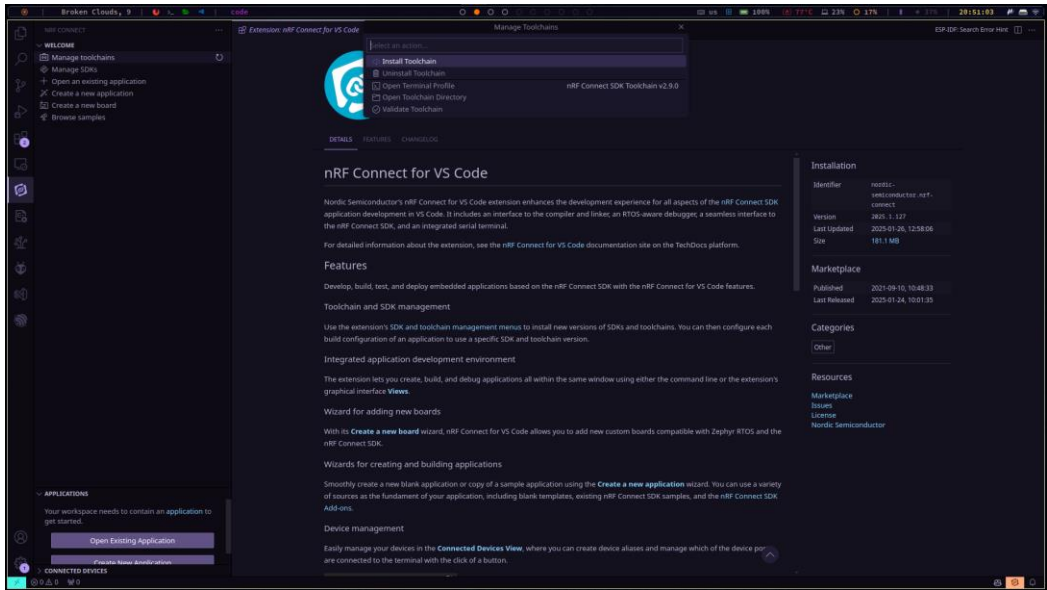
- **NCP (Network Co-Processor) Firmware**
 - The **nRF52840** runs OpenThread but offloads **network stack processing** to a host (e.g., a Linux system running wpantund).
 - The dongle communicates with the host via **UART or USB CDC**.
 - The host system handles higher-level networking tasks like **mesh routing and IPv6 processing**.
- **RCP (Radio Co-Processor) Firmware**
 - The nRF52840 runs only the **IEEE 802.15.4 MAC layer** and acts purely as a **radio device**.
 - The host (e.g., a Raspberry Pi running OpenThread Border Router) takes full control of Thread operations.
 - The host uses SPI/UART/USB to communicate with the RCP firmware.
 - This architecture is used in **low-power border routers**.

nRF52840 Firmware Guide

- First, install the nRF Connect for VS Code extension.

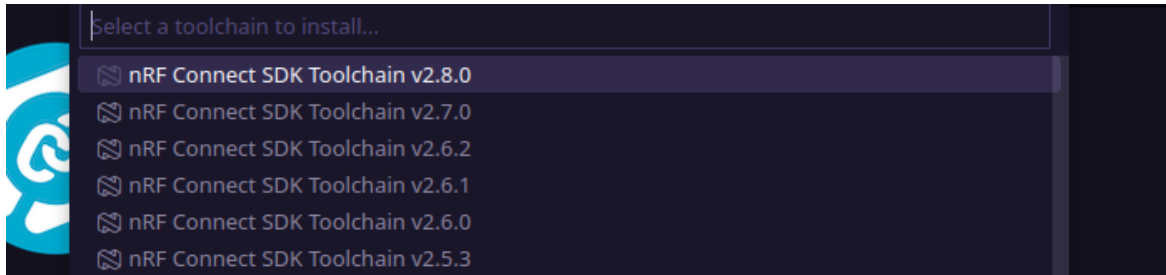


- Next, install the toolchain.

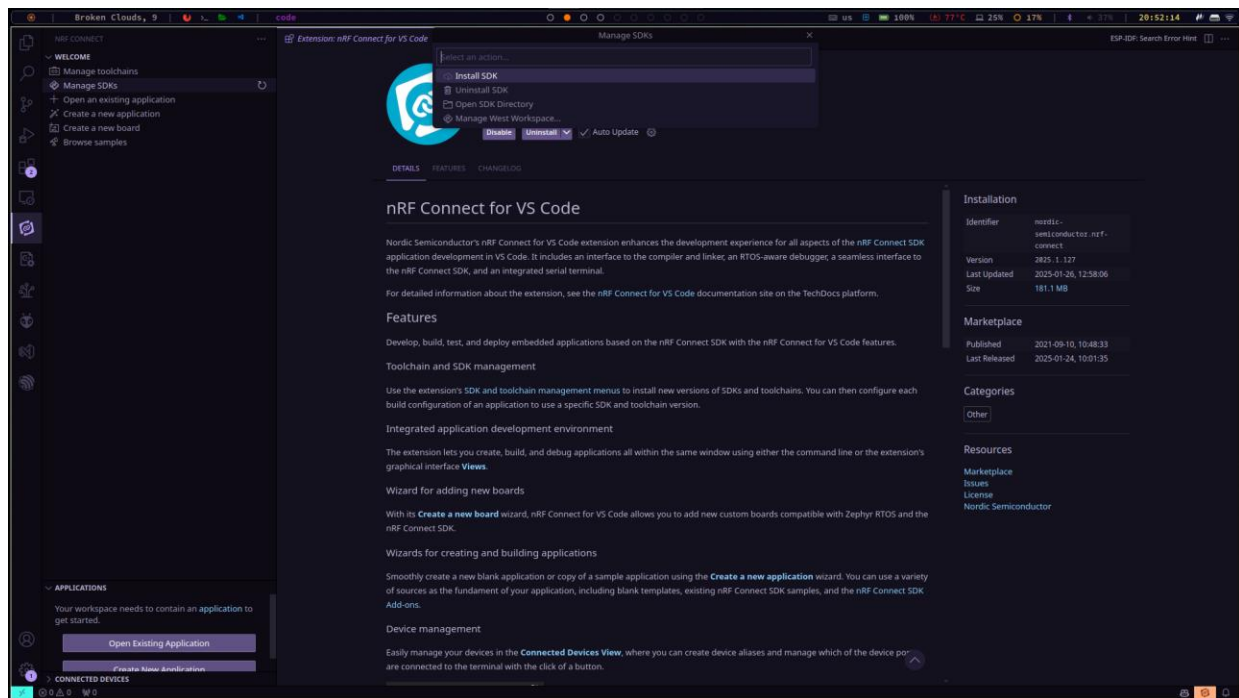


OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 11 / 29

- Select the newest available. In my case I run the v2.9.0, but since it is already installed, it doesn't show, but the extension allows the installation of multiple toolchains for version control.

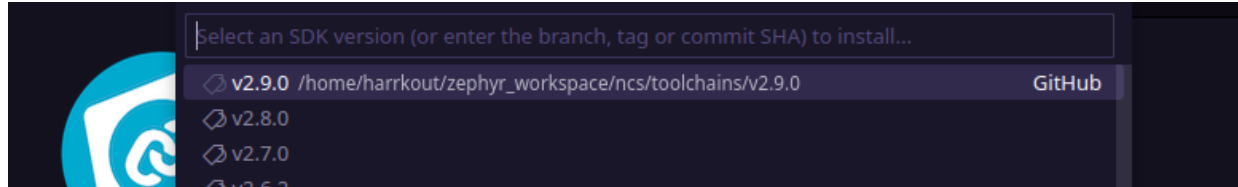


- Next, install the SDK.

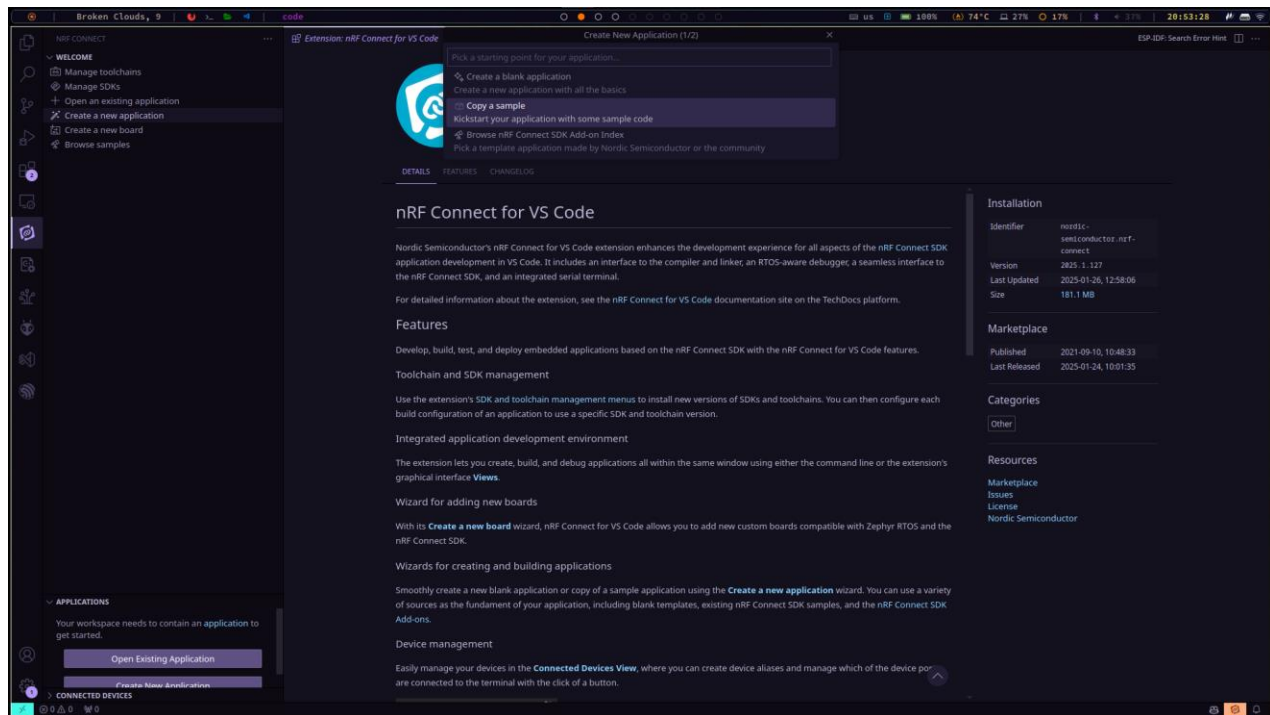


OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 12 / 29

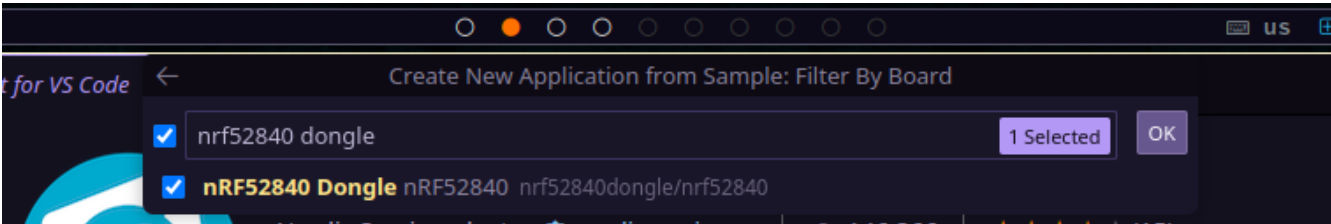
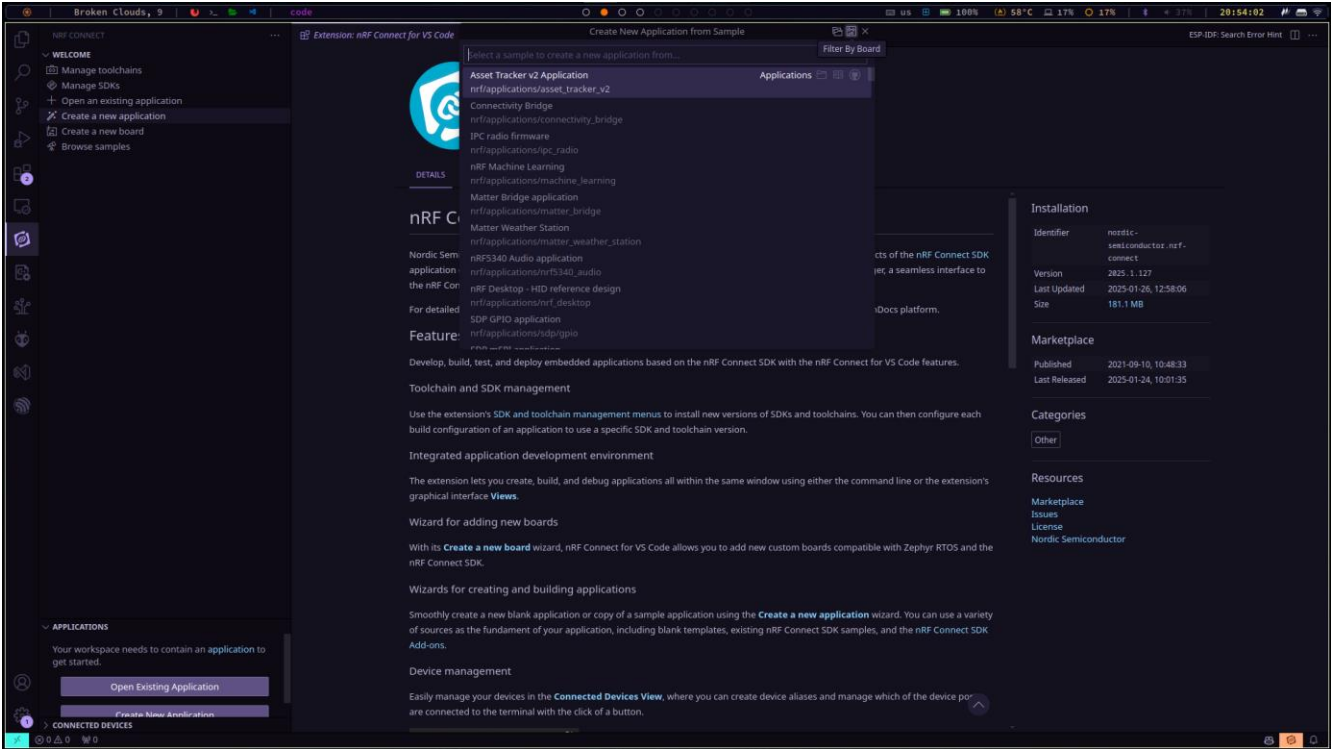
- Next, select the newest available. I have installed the v.2.9.0



- After the toolchain and SDK are installed correctly, we can browse the nRF SDK's applications, which can be found also here:
 - <https://github.com/nrfconnect/sdk-nrf/tree/main/samples>

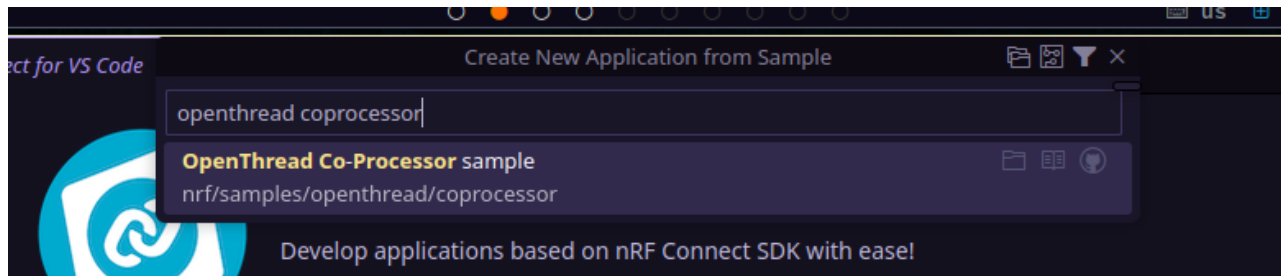


- Next, in order to find which applications are compatible out-of-the-box with a specific board, we can select the option **Filter By Board**, and search for our device. In this case we search for **nRF52840**.

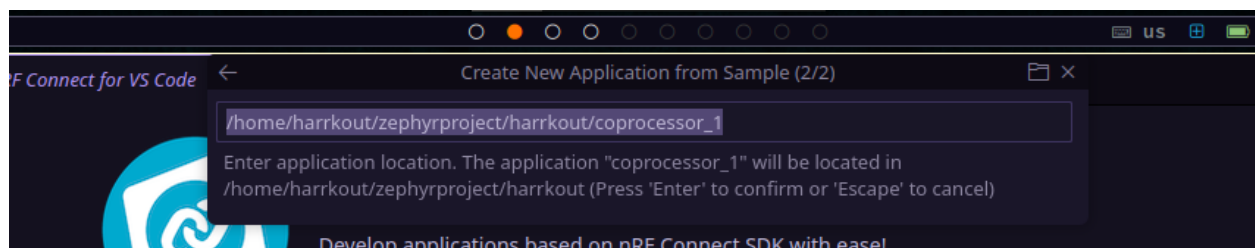


OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 14 / 29

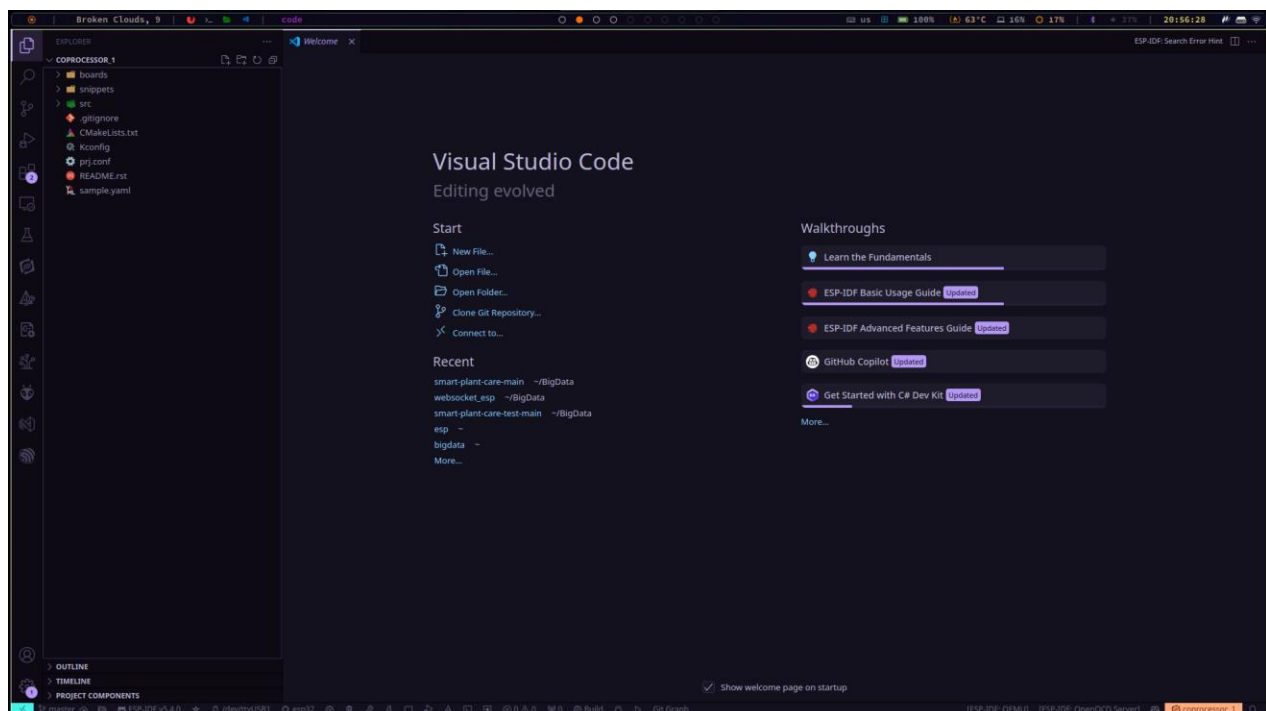
- And now we can search for the **Co-Processor** application.



- And create the new application's directory in the suggested folder, note that it needs to be inside the installed Zephyr's directory.

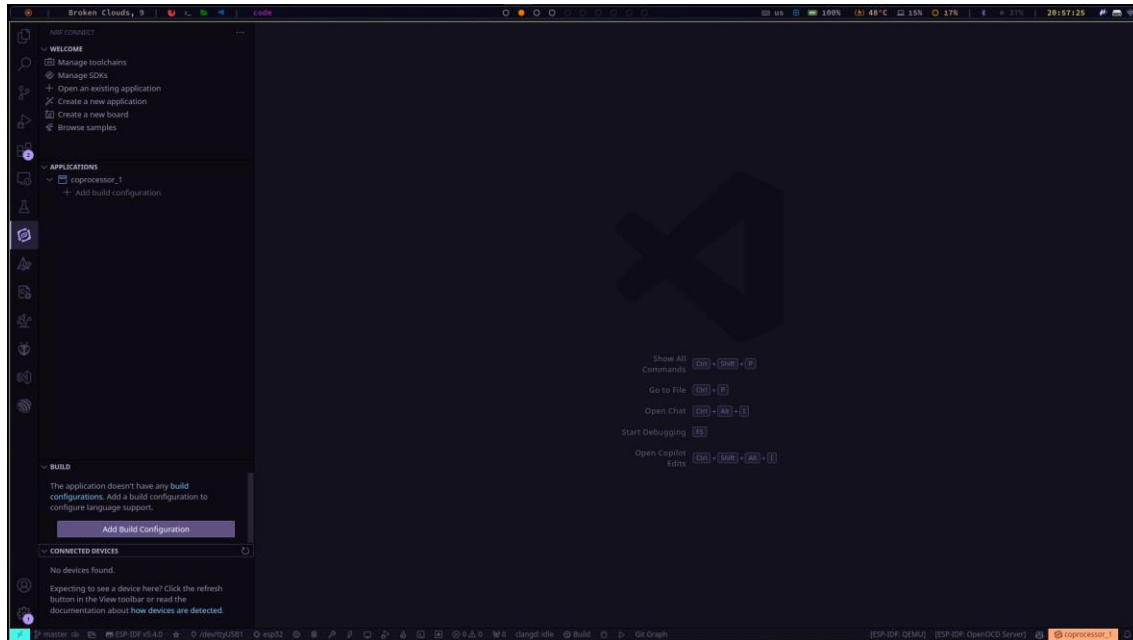


- And like that, our application has been created, or rather copied from the SDK database, into our directory.



OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 15 / 29

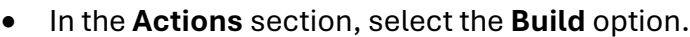
- Now, let's go back to the nRF Connect extension, where we can see the **coprocessor** application in the **Applications** section.



- Now we need to create the **Build Configuration**, for our board, and select the option **Build** at the bottom. Just selectin the board is enough for now, no further options are needed.

OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 16 / 29

Page 16 / 29

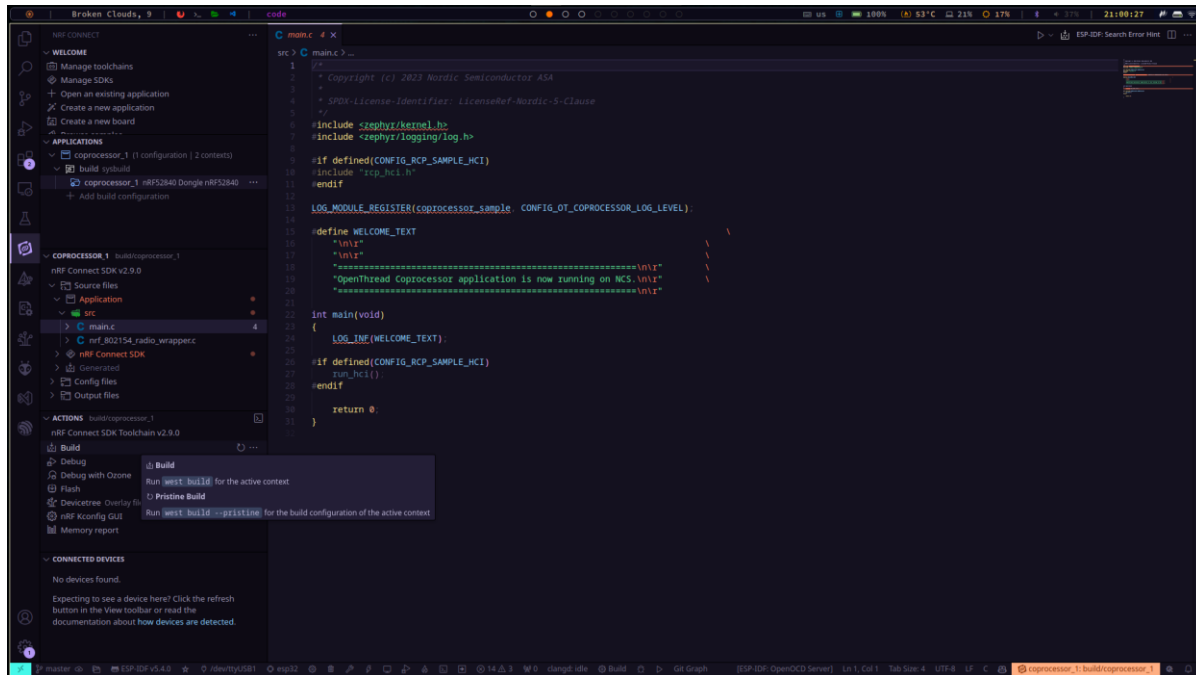


OpenThread Guide

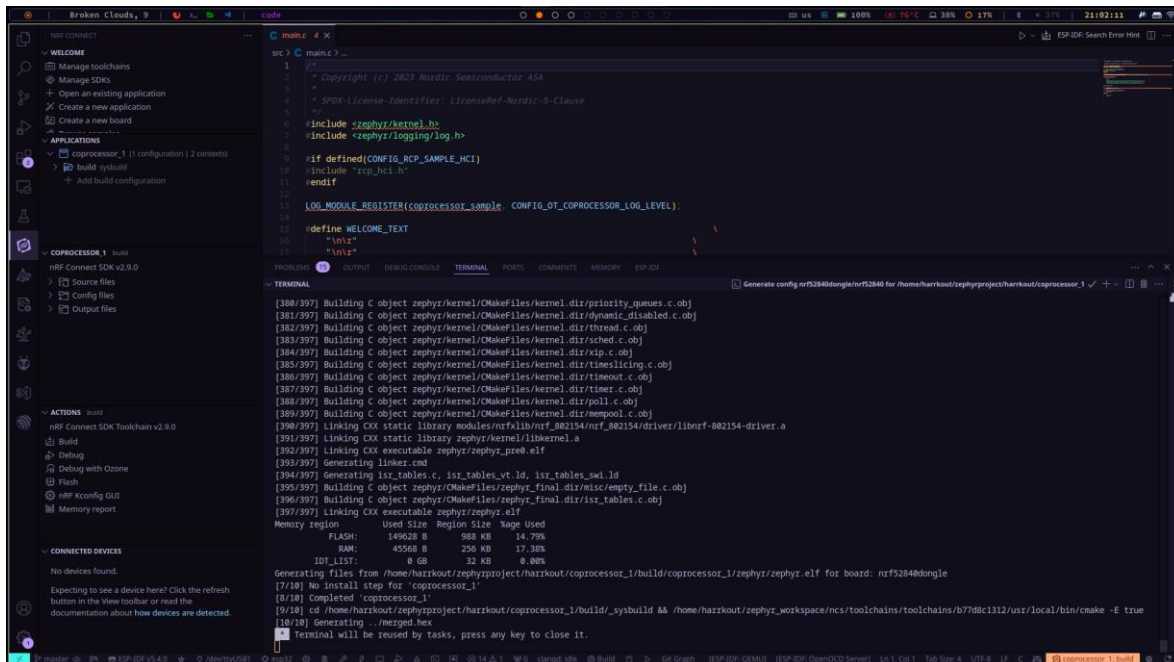
Doc # DLV1.1

Version: 1.1

Page 17 / 29



- Open the Terminal (Ctrl + `) to watch the status of the build.



OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 18 / 29

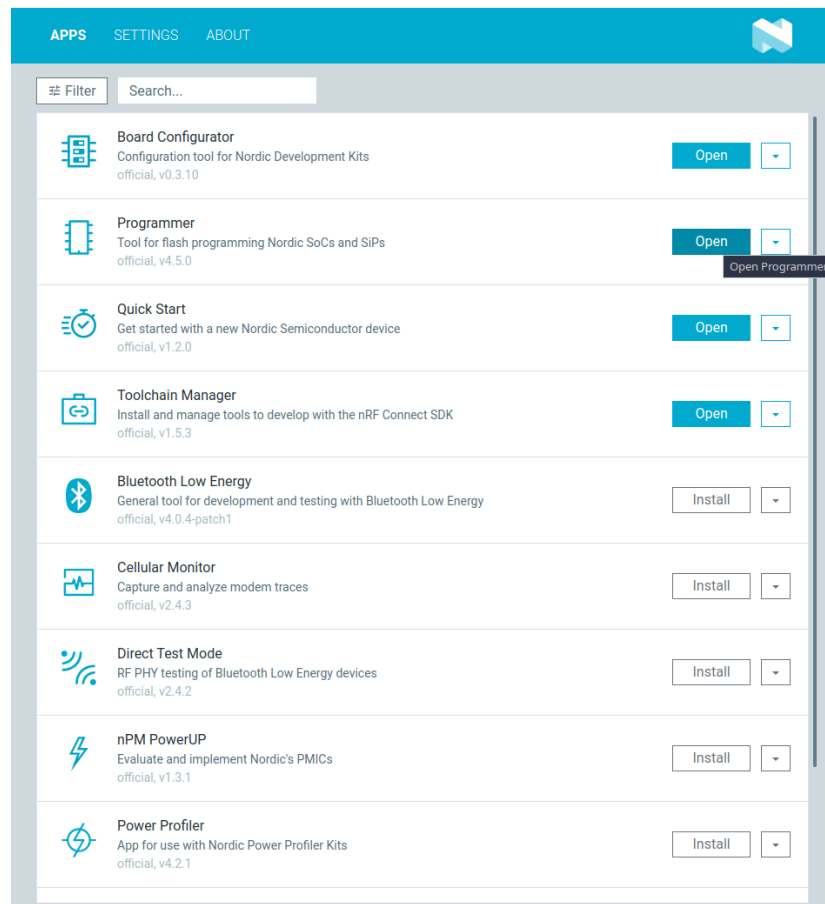
- After the build has completed successfully, browse to the application directory/**build**, to ensure the firmware has compiled successfully. The target file is the **merged.hex**.

```

1 - harrkout@rth: ~/zephyrproject/harrkout/coprocessor_1/build
> ls
├── _sysbuild
├── build.ninja
├── build_info.yml
├── build_info.yml.bak
├── cmake_install.cmake
├── CMakeCache.txt
├── CMakeFiles
├── coprocessor_1
├── coprocessor_1_sysbuild_cache.txt
├── coprocessor_1_sysbuild_cache.txt.tmp
├── domains.yml
├── Kconfig
├── merged.hex
├── modules
├── partitions.yml
├── pm.config
├── regions.yml
├── sysbuild_modules.txt
├── zephyr
├── zephyr_modules.txt
└── zephyr_settings.txt

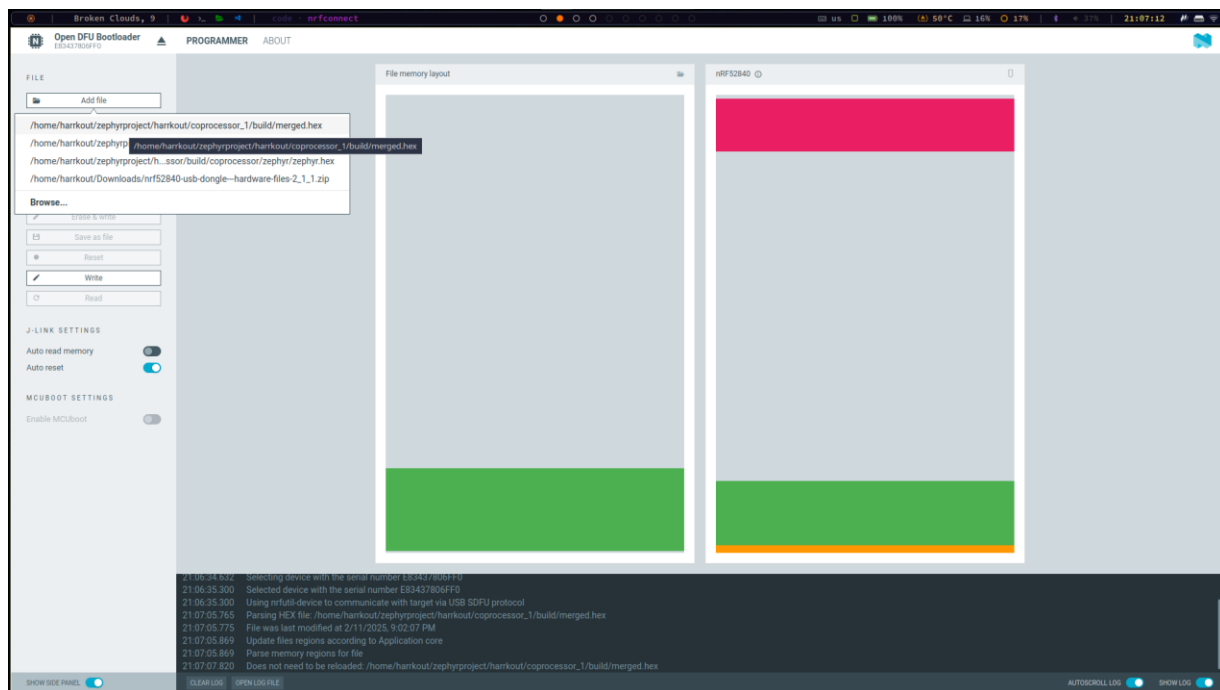
```

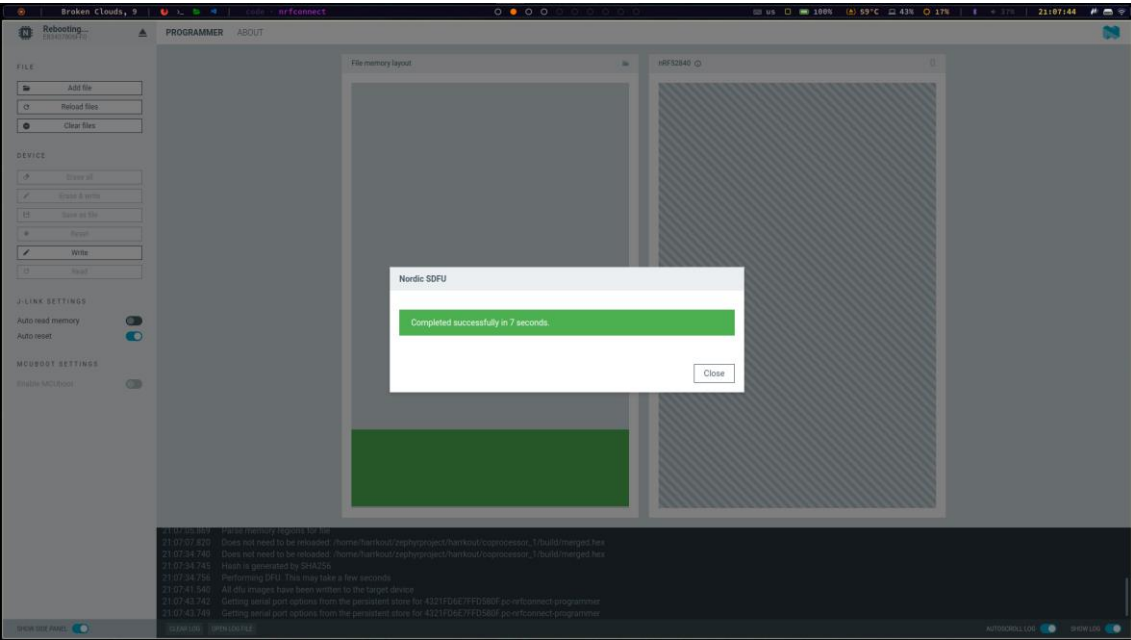
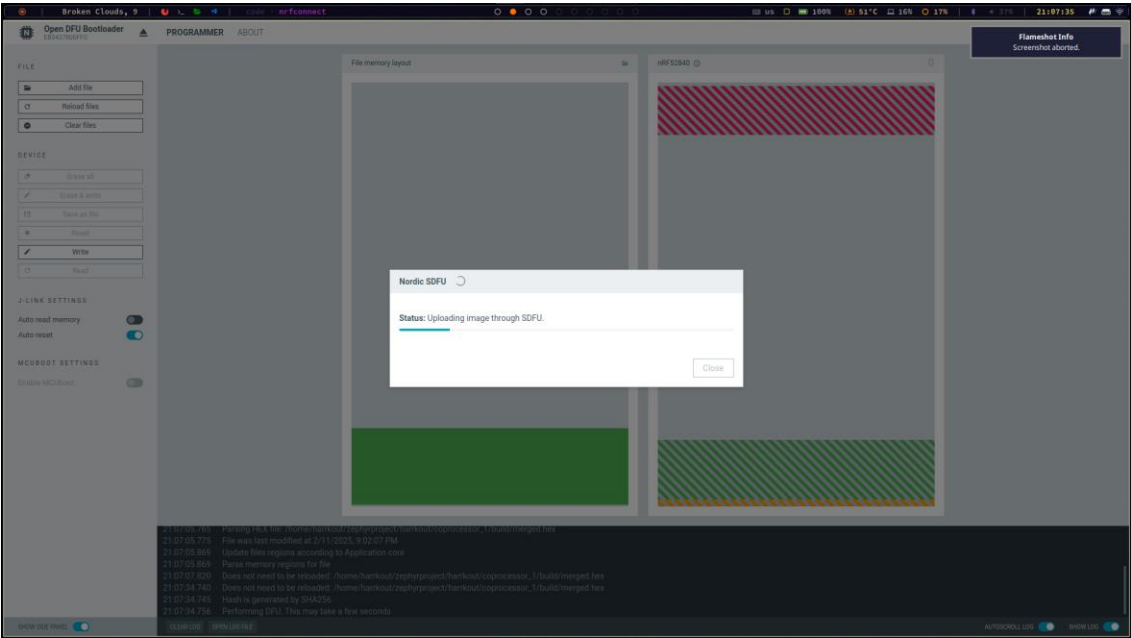
- Now, we are ready to flash the firmware to the nRF52840 dongle. To do that, we need the nRF Connect Desktop App (see **Tools** section), and then select the **Programmer**.



OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 19 / 29

- Now, before we proceed to the next image, we need to connect the nRF52840 dongle to the host machine, but, since the device does not have an on-board debugger, we need to put the device in **DFU (Device Firmware Update) mode**. To do that, press the **Reset** button (sideways to the white button – note that there are 2 buttons in the device) while plugging the device to the USB port.
- After the device has been plugged, select it from the top left corner. If the device is not in DFU mode, an error will show.
- Then select the **merged.bin** file from the build directory.
- And finally, press **Write**.





OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 21 / 29

- Then, unplug the device, plug it again and check the UART for the following messages.
- If the following messages are shown, we have successfully flashed the **Co-Processor** firmware to the nRF52840 dongle.

```
> cat /dev/ttyACM0
*** Booting nRF Connect SDK v2.9.0-7787b2649840 ***

*** Using Zephyr OS v3.7.99-1f8f3dc29142 ***
```

OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 22 / 29

nRF7002DK & nRF5340DK Firmware

In the same manner, with the VSCode extension, the firmware for the other nRF boards is built, with the difference that they can be flashed via the extension, since they house an integrated on-board debugger.

Both board will run the same application, the **OpenThread CLI**.

- <https://github.com/nrfconnect/sdk-nrf/tree/main/samples/openthread/cli>

What is OpenThread CLI Firmware?

- It provides a **direct interface** to the OpenThread stack for testing, debugging, and configuring a Thread network.
- It runs **entirely on the board**, meaning **no external host is needed**.
- You interact with it through a **serial terminal** (e.g., PuTTY, minicom, or screen).

Key Features of OpenThread CLI Firmware

- **Thread Network Control**
 - Create a Thread network using commands like **dataset commit active**, **ifconfig up**, and **thread start**.
 - Join an existing Thread network with **joiner start**.
 - Configure network settings such as **PAN ID**, **channel**, and **network key**.
- **Communication & Addressing**
 - Send and receive UDP packets using **udp open** and **udp send**.
 - Manage IPv6 addresses with the **ipaddr command**.
- **Device Roles**
 - Operate as a **Full Thread Device (FTD)** or a **Minimal Thread Device (MTD)**.

OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 23 / 29

- Set the device role to **Leader, Router, or End Device**.

- **Security & Keys**

- Manage network security settings, including **PSKc keys, network credentials, and access control**.

- **Energy & Radio Testing**

- Measure power consumption using the **power** command.
- Control the radio for transmission and reception using mac send and other radio commands.

CLI vs. NCP/RCP: What's the Difference?

Feature	CLI Firmware	NCP Firmware	RCP Firmware
Runs OpenThread Stack	✓ Yes	✓ Yes	✗ No (host controls Thread)
Needs External Host	✗ No	✓ Yes (e.g., Linux host)	✓ Yes (e.g., OTBR host)
Serial Interface	USB CDC/ UART	UART/USB	UART/SPI
Used for Debugging	✓ Yes	✗ No	✗ No
Used in OpenThread Border Router	✗ No	✓ Yes	✓ Yes

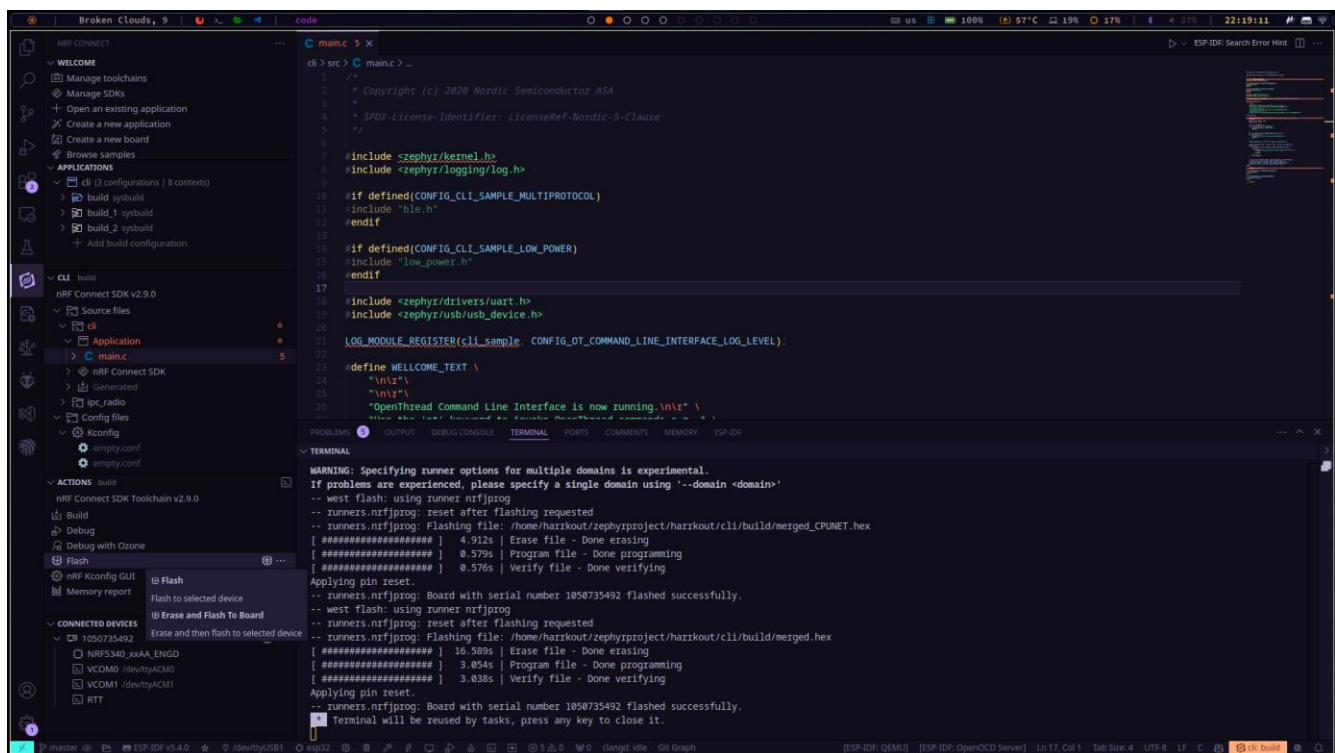
OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 24 / 29

Build and Flash

The process is the same as shown previously, except for choosing the boards

- **nrf7002dk/nrf5340/cpuapp** for the **nRF7002DK** board, and
- **nrf5340dk/nrf5340/cpuapp** for the **nRF5340DK** board.

And selecting the application **OpenThread CLI Sample**.



OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 25 / 29

OpenThread Border Router Setup

In order to setup the OpenThread network, we need a host machine (Raspberry Pi or other Linux machine), to run the OpenThread Border Router application.

The OpenThread Border Router (OTBR) is a gateway between a Thread network and other networks, such as Wi-Fi or Ethernet. It allows Thread devices to communicate with external networks like the internet.

Features of OTBR

Thread to IP Bridging

- Connects a Thread network to a wider IP network like Wi-Fi or Ethernet.
- Allows communication between Thread and non-Thread devices.

Border Agent & Thread Commissioning

- Enables secure onboarding of new Thread devices.
- Supports commissioning via Bluetooth LE or external Joiners.

Routing & Connectivity

- Functions as a full Thread router.
- Can act as a Leader or Router in the Thread network.

DNS-Based Service Discovery

- Provides local service discovery for Thread devices.
- Uses mDNS (Multicast DNS) and SRP (Service Registration Protocol).

Integration with IoT Protocols

- Supports IoT communication through MQTT, Matter, and CoAP.

OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 26 / 29

There are two options in running the OTBR in the host machine:

- Setting manually, with strict packet version control of any dependent application, or
- In a docker container, which is what is used here, for ease of use.

A fully detailed guide run the OTBR in a docker container can be found here:

- <https://openthread.io/guides/border-router/docker/run>

On the host machine, run the command (based on linux distribution, this is being run on Manjaro Linux):

```
docker run --sysctl "net.ipv6.conf.all.disable_ipv6=0 net.ipv4.conf.all.forwarding=1
net.ipv6.conf.all.forwarding=1" -p 8080:80 --dns=127.0.0.1 -it --volume
/dev/ttyACM0:/dev/ttyACM0 --privileged openthread/otbr --radio-url
spinel+hdlc+uart:///dev/ttyACM0
```

OpenThread Guide		
Doc # DLV1.1	Version: 1.1	Page 27 / 29

OT Border Router

Home

Home

Join

Form

Status

Settings

Commission

Topology

OPEN⁺THREAD

released by Google

What is OpenThread?

...an open-source implementation of the Thread networking protocol. Nest has released OpenThread to make the technology used in Nest products more broadly available to developers to accelerate the development of products for the connected home.

...OS and platform agnostic, with a narrow platform abstraction layer and a small memory footprint, making it highly portable.

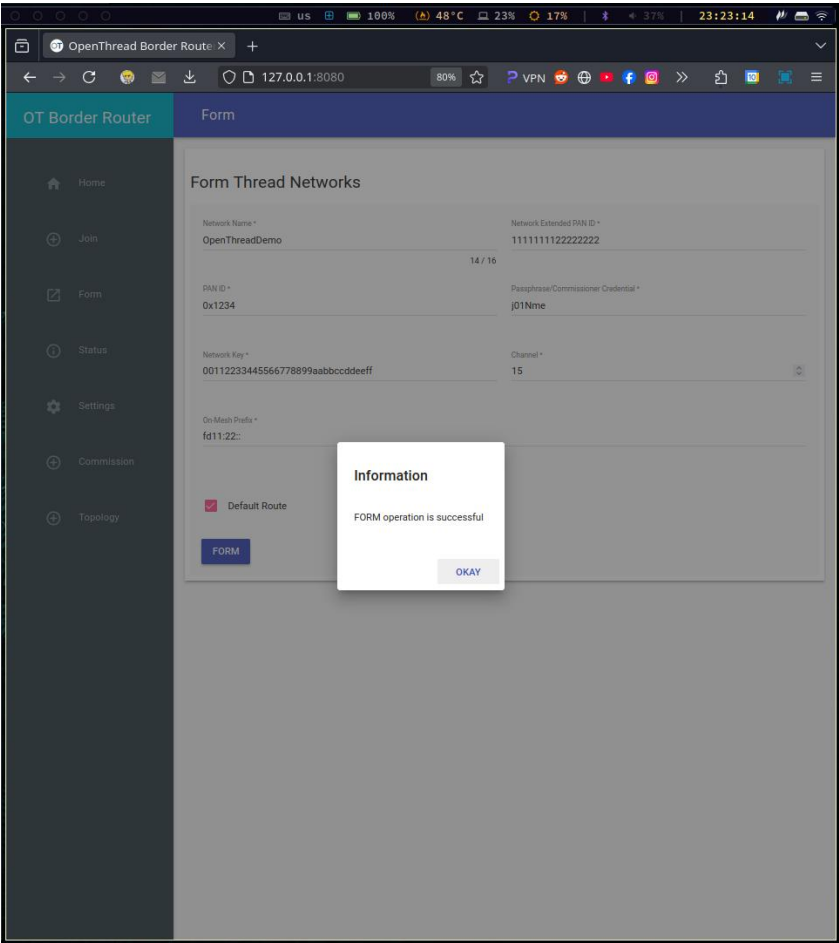
...a Thread Certified Component implementing all features defined in the Thread 1.1.1 specification. This specification defines an IPv6-based reliable, secure and low-power wireless device-to-device communication protocol for home applications.

What is OpenThread Border Router?

In the context of a Thread Network, a Border Router is a device that provides connectivity of nodes in the Thread Network to other devices in external networks such as the wider Internet, local home and building IP networks, or virtual private networks (Figure 1).

```
graph LR; Device[Mobile Device] ---|Wi-Fi or Ethernet| Cloud[Cloud]; Cloud ---|Wi-Fi or Ethernet| BR[Border Router]; BR ---|Thread Network| Mesh[Thread Network Mesh]
```

Figure 9-1. High Level Overview of Border Router Role



OpenThread Guide

Doc # DLV1.1

Version: 1.1

Page 29 / 29

```
Feb 11 21:21:38 8a5cfcd864f otnr-agent[199]: 00:00:22.570 [I] P-Netif-----: ADD [M] ff03::2 (already subscribed, ignored)
Feb 11 21:21:38 8a5cfcd864f otnr-agent[199]: 00:00:22.570 [I] P-Netif-----: ADD [M] ff35:40:f0d1:597d:28bc:cd5d:0:1 (already subscribed, ignored)
Feb 11 21:21:38 8a5cfcd864f otnr-agent[199]: 00:00:22.570 [I] P-Netif-----: ADD [M] ff32:40:f0d1:597d:28bc:cd5d:0:1 (already subscribed, ignored)
Feb 11 21:21:38 8a5cfcd864f otnr-agent[199]: 00:00:22.570 [I] P-Netif-----: ADD [M] ff03::fc (already subscribed, ignored)
Feb 11 21:21:38 8a5cfcd864f otnr-agent[199]: 00:00:22.570 [I] P-Netif-----: ADD [M] ff03::1 (already subscribed, ignored)
Feb 11 21:21:38 8a5cfcd864f otnr-agent[199]: 00:00:22.570 [I] P-Netif-----: ADD [M] ff02::16 (already subscribed, ignored)
Feb 11 21:21:38 8a5cfcd864f otnr-agent[199]: 00:00:22.570 [I] P-Netif-----: ADD [M] ff02::2 (already subscribed, ignored)
Feb 11 21:21:38 8a5cfcd864f otnr-agent[199]: 00:00:22.570 [I] P-Netif-----: ADD [M] ff03::2 (already subscribed, ignored)
Feb 11 21:21:38 8a5cfcd864f otnr-agent[199]: 00:00:22.570 [I] P-Netif-----: ADD [M] ff35:40:f0d1:597d:28bc:cd5d:0:1 (already subscribed, ignored)
Feb 11 21:21:38 8a5cfcd864f otnr-agent[199]: 00:00:22.570 [I] P-Netif-----: ADD [M] ff32:40:f0d1:597d:28bc:cd5d:0:1 (already subscribed, ignored)
Feb 11 21:21:38 8a5cfcd864f otnr-agent[199]: 00:00:22.570 [I] P-Netif-----: ADD [M] ff03::fc (already subscribed, ignored)
Feb 11 21:21:38 8a5cfcd864f otnr-agent[199]: 00:00:22.570 [I] P-Netif-----: ADD [M] ff03::1 (already subscribed, ignored)
Feb 11 21:21:38 8a5cfcd864f otnr-agent[199]: 00:00:22.570 [I] P-Netif-----: ADD [M] ff02::16 (already subscribed, ignored)
Feb 11 21:21:38 8a5cfcd864f otnr-agent[199]: 00:00:22.570 [I] P-Netif-----: ADD [M] ff02::2 (already subscribed, ignored)
Feb 11 21:21:39 8a5cfcd864f otnr-agent[199]: 00:00:23.357 [I] Mle-----: Send Advertisement (ff02:0:0:0:0:0:1)
Feb 11 21:21:39 8a5cfcd864f otnr-agent[199]: 00:00:23.358 [D] P-SpinelDrive: Sent spinel frame, flg:0x2, iid:0, tid:11, cmd:PROP_VALUE_SET, key:STREAM_RAW, Len:69, channel:15, maxbackoffs:4, maxretries:15 ...
Feb 11 21:21:39 8a5cfcd864f otnr-agent[199]: 00:00:23.358 [D] P-SpinelDrive: ... cameCaEnabled:1, isHeaderUpdated:0, isARetx:0, skipAs:0, txDelay:0, txDelayBase:0
Feb 11 21:21:39 8a5cfcd864f otnr-agent[199]: 00:00:23.364 [D] P-SpinelDrive: Received spinel frame, flg:0x2, iid:0, tid:11, cmd:PROP_VALUE_IS, key:LAST_STATUS, status:OK
Feb 11 21:21:39 8a5cfcd864f otnr-agent[199]: 00:00:23.364 [I] MeshForwarder: Sent IPv6 UDP msg, Len:90, chksum:dafo, ecn:no, to:0xffff, sec:no, prio:net
Feb 11 21:21:39 8a5cfcd864f otnr-agent[199]: 00:00:23.364 [I] MeshForwarder: src:[fe80:0:0:0:bc64:3551:6fd6:e0d7]:19788
Feb 11 21:21:39 8a5cfcd864f otnr-agent[199]: 00:00:23.364 [I] MeshForwarder: dst:[ff02:0:0:0:0:0:0:1]:19788
Feb 11 21:21:40 8a5cfcd864f otnr-agent[199]: 00:00:24.179 [I] RoutingManager: Evaluating routing policy
Feb 11 21:21:40 8a5cfcd864f otnr-agent[199]: 00:00:24.179 [I] RoutingManager: Evaluating NAT64 prefix
Feb 11 21:21:40 8a5cfcd864f otnr-agent[199]: 00:00:24.179 [I] RoutingManager: Preparing RA
Feb 11 21:21:40 8a5cfcd864f otnr-agent[199]: 00:00:24.179 [I] RoutingManager: RA Header - fFlags - M:0 O:0 S:1
Feb 11 21:21:40 8a5cfcd864f otnr-agent[199]: 00:00:24.179 [I] RoutingManager: RA Header - default route - Lifetime:0
Feb 11 21:21:40 8a5cfcd864f otnr-agent[199]: 00:00:24.179 [I] RoutingManager: - PIO fd11:1111:1122:2222::/64 (valid:1800, preferred:1800)
Feb 11 21:21:40 8a5cfcd864f otnr-agent[199]: 00:00:24.179 [I] RoutingManager: - RIO fd11:22:0:0::/64 (Lifetime:1800, prf:medium)
Feb 11 21:21:40 8a5cfcd864f otnr-agent[199]: 00:00:24.179 [I] RoutingManager: Sent RA on infra netif 13
Feb 11 21:21:40 8a5cfcd864f otnr-agent[199]: 00:00:24.179 [I] RoutingManager: Will evaluate routing policy in 00:15.887 (15887 msec)
Feb 11 21:21:40 8a5cfcd864f otnr-agent[199]: 00:00:24.180 [I] RoutingManager: Received RA from fe80:0:0:0:42:acff:f011:2 on infra netif 13 (this BR routing-manager)
Feb 11 21:21:48 8a5cfcd864f otnr-agent[199]: 00:00:31.795 [I] Mle-----: Send Advertisement (ff02:0:0:0:0:0:0:1)
Feb 11 21:21:48 8a5cfcd864f otnr-agent[199]: 00:00:31.795 [D] P-SpinelDrive: Sent spinel frame, flg:0x2, iid:0, tid:12, cmd:PROP_VALUE_SET, key:STREAM_RAW, Len:69, channel:15, maxbackoffs:4, maxretries:15 ...
Feb 11 21:21:48 8a5cfcd864f otnr-agent[199]: 00:00:31.795 [D] P-SpinelDrive: ... cameCaEnabled:1, isHeaderUpdated:0, isARetx:0, skipAs:0, txDelay:0, txDelayBase:0
Feb 11 21:21:48 8a5cfcd864f otnr-agent[199]: 00:00:31.799 [D] P-SpinelDrive: Received spinel frame, flg:0x2, iid:0, tid:12, cmd:PROP_VALUE_IS, key:LAST_STATUS, status:OK
Feb 11 21:21:48 8a5cfcd864f otnr-agent[199]: 00:00:31.799 [I] MeshForwarder: Sent IPv6 UDP msg, Len:90, chksum:43f2, ecn:no, to:0xffff, sec:no, prio:net
Feb 11 21:21:48 8a5cfcd864f otnr-agent[199]: 00:00:31.799 [I] MeshForwarder: src:[fe80:0:0:0:bc64:3551:6fd6:e0d7]:19788
Feb 11 21:21:48 8a5cfcd864f otnr-agent[199]: 00:00:31.799 [I] MeshForwarder: dst:[ff02:0:0:0:0:0:0:1]:19788
Feb 11 21:21:56 8a5cfcd864f otnr-agent[199]: 00:00:40.066 [I] RoutingManager: Evaluating routing policy
Feb 11 21:21:56 8a5cfcd864f otnr-agent[199]: 00:00:40.066 [I] RoutingManager: Evaluating NAT64 prefix
Feb 11 21:21:56 8a5cfcd864f otnr-agent[199]: 00:00:40.066 [I] RoutingManager: Preparing RA
Feb 11 21:21:56 8a5cfcd864f otnr-agent[199]: 00:00:40.066 [I] RoutingManager: RA Header - fFlags - M:0 O:0 S:1
Feb 11 21:21:56 8a5cfcd864f otnr-agent[199]: 00:00:40.066 [I] RoutingManager: RA Header - default route - Lifetime:0
Feb 11 21:21:56 8a5cfcd864f otnr-agent[199]: 00:00:40.066 [I] RoutingManager: - PIO fd11:1111:1122:2222::/64 (valid:1800, preferred:1800)
Feb 11 21:21:56 8a5cfcd864f otnr-agent[199]: 00:00:40.066 [I] RoutingManager: - RIO fd11:22:0:0::/64 (Lifetime:1800, prf:medium)
Feb 11 21:21:56 8a5cfcd864f otnr-agent[199]: 00:00:40.066 [I] RoutingManager: Sent RA on infra netif 13
Feb 11 21:21:56 8a5cfcd864f otnr-agent[199]: 00:00:40.066 [I] RoutingManager: Will evaluate routing policy in 00:16.472 (16472 msec)
Feb 11 21:21:56 8a5cfcd864f otnr-agent[199]: 00:00:40.067 [I] RoutingManager: Received RA from fe80:0:0:0:42:acff:f011:2 on infra netif 13 (this BR routing-manager)
```